

BDA Lab 2: HDFS File Operations using Java FileSystem API

Simple lab-manual steps with a basic Java program for writing, reading, seeking, and listing files on HDFS.

Aim

To use the Hadoop FileSystem API to create a file, write data, read data, seek to a particular position, and list files stored in HDFS.

1. Create the working folder

Run these commands in Windows PowerShell:

```
cd C:\bda-labs
mkdir lab2-hdfs
cd lab2-hdfs
```

2. Create the Docker Compose file

Create a file named docker-compose.yml using Notepad:

```
notepad docker-compose.yml
```

Paste and save the following content:

```
services:
  namenode:
    image: apache/hadoop:3.3.6
    hostname: namenode
    command: ["hdfs", "namenode"]
    ports:
      - "9870:9870"
    env_file: ./config
    environment:
      ENSURE_NAMENODE_DIR: "/tmp/hadoop-root/dfs/name"

  datanode:
    image: apache/hadoop:3.3.6
    command: ["hdfs", "datanode"]
    env_file: ./config
```

3. Create the configuration file

Create another file named config in the same folder:

```
notepad config
```

Paste and save:

```
CORE-SITE.XML_fs.default.name=hdfs://namenode
CORE-SITE.XML_fs.defaultFS=hdfs://namenode
HDFS-SITE.XML_dfs.namenode.rpc-address=namenode:8020
HDFS-SITE.XML_dfs.replication=1
```

4. Start HDFS

```
docker compose up -d
```

```
docker compose exec namenode bash
```

Inside the namenode container, create a directory in HDFS:

```
hdfs dfs -mkdir -p /user/student
hdfs dfs -ls /user
```

5. Create the Java program

Inside the namenode container, create HdfsDemo.java:

```
nano HdfsDemo.java
```

Basic Java code for writing and understanding:

```
import java.net.URI;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.FileStatus;
import org.apache.hadoop.fs.FileSystem;
import org.apache.hadoop.fs.FSDataInputStream;
import org.apache.hadoop.fs.FSDataOutputStream;
import org.apache.hadoop.fs.Path;

public class HdfsDemo {

    public static void main(String[] args) throws Exception {

        String uri = "hdfs://namenode:8020";

        Configuration conf = new Configuration();
        FileSystem fs = FileSystem.get(URI.create(uri), conf);

        Path file = new Path("/user/student/quangle.txt");

        // Write data to HDFS
        FSDataOutputStream out = fs.create(file);
        out.writeBytes("On the top of the Crumpetty Tree\n");
        out.writeBytes("The Quangle Wangle sat\n");
        out.close();

        System.out.println("File written successfully");

        // Read data from HDFS
        FSDataInputStream in = fs.open(file);

        byte[] buffer = new byte[32];
        in.read(buffer);
        System.out.println("From start: " + new String(buffer).trim());

        // Move to a particular byte position
        in.seek(33);

        byte[] buffer2 = new byte[32];
        in.read(buffer2);
        System.out.println("After seek: " + new String(buffer2).trim());

        in.close();

        // Display file information
        FileStatus status = fs.getFileStatus(file);
        System.out.println("File length: " + status.getLen());
        System.out.println("Replication: " + status.getReplication());

        // List files in a directory
        System.out.println("Files in /user/student:");
        for (FileStatus s : fs.listStatus(new Path("/user/student"))) {
            System.out.println(s.getPath().getName());
        }

        // Find text files using a wildcard
        System.out.println("Text files:");
        for (FileStatus s : fs.globStatus(new Path("/user/student/*.txt"))) {
            System.out.println(s.getPath().getName());
        }

        fs.close();
    }
}
```

Save in nano using: Ctrl + O, Enter, Ctrl + X.

6. Compile and create the JAR file

```
export HADOOP_CLASSPATH=$(hadoop classpath)
```

```
mkdir classes
```

```
javac -classpath $HADOOP_CLASSPATH -d classes HdfsDemo.java
```

```
jar -cvf hdfsdemo.jar -C classes .
```

The program is compiled using Hadoop libraries and packaged into hdfsdemo.jar.

7. Run the Java program

```
hadoop jar hdfsdemo.jar HdfsDemo
```

Expected output:

```
File written successfully
From start: On the top of the Crumpetty Tree
After seek: The Quangle Wangle sat
File length: 56
Replication: 1
Files in /user/student:
quangle.txt
Text files:
quangle.txt
```

8. Verify the file directly in HDFS

```
hdfs dfs -cat /user/student/quangle.txt
```

9. Add another file and run again

```
echo "hello hdfs" | hdfs dfs -put - /user/student/notes.txt
```

```
hadoop jar hdfsdemo.jar HdfsDemo
```

The listStatus and globStatus sections should now show both quangle.txt and notes.txt.

10. Important concepts

create() writes a file to HDFS. **open()** reads a file. **seek()** moves to an absolute byte position while reading. **getFileStatus()** displays file metadata. **listStatus()** lists files in a directory. **globStatus()** lists files matching a pattern such as *.txt.

11. Clean up

```
exit
```

Back in Windows PowerShell:

```
docker compose down
```